

Cloud Computing Tools

Deliverable 2: Verificación y consolidación de datos meteorológicos en JupyterHub sobre AWS EC2

Juan Pablo Jiménez — Maestría en Analítica Aplicada
Harrison Alonso — Maestría en Analítica Aplicada
Juan Fernando Serrano — Maestría en Analítica Aplicada
Universidad de La Sabana
Maestría en Analítica Aplicada
Prof.: Hugo Franco, Ph.D.
26 de agosto de 2026

Código fuente: <https://github.com/juanferrrr/Deliverable-2-Grupo-Alonso-Jimenez-Serrano>

1. Verificación de la ingesta (Task A)

1.1. Descripción del problema

El servidor de ingesta (t2.micro) deposita cada hora, mediante un cron job configurado en el taller anterior, una lectura meteorológica en la zona cruda del Data Lake (s3://<bucket>/raw_zone/weather/). Antes de consolidar esas lecturas para el análisis, es necesario verificar programáticamente —y no solo confiar en la ausencia de errores del cron— que los objetos existen en S3 y que provienen de la fuente esperada.

1.2. Método de solución

Se crea un cliente de boto3 sin credenciales explícitas: el rol asociado (LabInstanceProfile) las expone a través de los metadatos de la instancia. Se usa un paginador sobre list_objects_v2 con el prefijo raw_zone/weather/, se descartan los objetos "carpeta" (Key terminada en /) y se listan los archivos restantes ordenados por fecha de modificación, junto con su tamaño.

```
s3 = boto3.client("s3")
creds = boto3.Session().get_credentials()
print("Método de credenciales:", creds.method) # iam-role

paginator = s3.get_paginator("list_objects_v2")
for page in paginator.paginate(Bucket=BUCKET, Prefix=PREFIX):
    for obj in page.get("Contents", []):
        if not obj["Key"].endswith("/"):
            archivos.append(obj)
```

1.3. Resultados

La ejecución confirma method = "iam-role" (sin credenciales embebidas) y región us-east-1. El listado devolvió 4 archivos, evidencia directa de que el cron job del taller anterior sí depositó lecturas de forma automática:

Fecha (UTC)	Tamaño	Objeto (Key)
2026-08-22 16:25:27	120 B	raw_zone/weather/weather-data_20260822162526.csv
2026-08-26 01:35:06	119 B	raw_zone/weather/weather-data_20260826013505.csv
2026-08-26 01:40:05	119 B	raw_zone/weather/weather-data_20260826014004.csv
2026-08-26 01:45:05	119 B	raw_zone/weather/weather-data_20260826014504.csv

Tabla 1: Inventario de raw_zone/weather/ obtenido con list_objects_v2. Fuente: elaboración propia.

2. Consolidación de datos meteorológicos (Task B)

2.1. Descripción del problema

El equipo de analítica requiere las lecturas en un único DataFrame, sin descargar los CSV a disco, para poder calcular estadísticos básicos previos al cruce con la afluencia de clientes.

2.2. Método de solución

Para cada Key devuelta por list_objects_v2 se invoca get_object, y el cuerpo de la respuesta (bytes) se envuelve con io.BytesIO para que pd.read_csv lo lea directamente en memoria. Los DataFrames parciales se acumulan en una lista y se unen con pd.concat(ignore_index=True). Sobre el DataFrame consolidado se calculan el número de lecturas y las estadísticas de temperature_C y humidity_pct.

```
s3_object = s3_client.get_object(Bucket=bucket_name, Key=key)
df = pd.read_csv(io.BytesIO(s3_object["Body"].read()))
dataframes.append(df)

consolidated_df = pd.concat(dataframes, ignore_index=True)
```

```
avg_temp      = consolidated_df["temperature_C"].mean()
min_humidity  = consolidated_df["humidity_pct"].min()
max_humidity  = consolidated_df["humidity_pct"].max()
```

2.3. Resultados

Con las 4 lecturas disponibles a la fecha de la corrida se obtuvo el siguiente resumen:

Métrica	Valor
Número de lecturas	4
Temperatura promedio (°C)	14.25
Humedad mínima (%)	62
Humedad máxima (%)	86

Tabla 2: Resumen de las lecturas consolidadas. Fuente: elaboración propia (salida de consolidated_df).

3. Ejecución FinOps

Cumpliendo el requisito obligatorio del taller, una vez capturada la evidencia de las Tasks A y B se detuvieron desde la consola de EC2 ambas instancias — el servidor de ingesta (t2.micro) y el servidor de JupyterHub (t3.medium) —, verificando en la columna Instance state que ambas quedaran en estado Stopped antes de cerrar la sesión de trabajo. La captura correspondiente se adjunta como evidencia en el notebook entregado.

4. Discusión y conclusiones

El resultado más relevante de este entregable no es el valor de las estadísticas en sí —la muestra de 4 lecturas es apenas suficiente para ilustrar el método, no para sostener un análisis de correlación con la afluencia de clientes— sino la verificación de que la arquitectura completa (cron en EC2 → S3 → JupyterHub) funciona de extremo a extremo sin intervención manual ni credenciales embebidas: todo el acceso a S3 se resolvió mediante el rol de instancia (LabInstanceProfile), tanto en el servidor de ingesta como en el de análisis. La consolidación en memoria, sin materializar los CSV en disco, mantiene al notebook desacoplado del almacenamiento local, lo cual es consistente con el patrón Medallion trabajado en el taller anterior. Como trabajo futuro, una vez el cron job acumule más lecturas por hora, valdría la pena repetir Task B para obtener una serie con suficiente varianza y así habilitar la correlación temperatura/humedad-afluencia que motiva este flujo de trabajo. Por último, la disciplina de FinOps aplicada —detener ambas instancias inmediatamente después de capturar la evidencia— es tan parte del entregable técnico como el propio código: en AWS Academy Learner Lab, un t3.medium olvidado corriendo agota el crédito disponible y bloquea el laboratorio de forma irreversible.